

Stats meth

Nicholas Bartolucci

2026-04-26

```
# Step 1: Library
```

```
library(MASS)
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.4.2
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following object is masked from 'package:MASS':
```

```
##
```

```
##      select
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      intersect, setdiff, setequal, union
```

```
library(quantmod)
```

```
## Warning: package 'quantmod' was built under R version 4.4.3
```

```
## Loading required package: xts
```

```
## Warning: package 'xts' was built under R version 4.4.3
```

```
## Loading required package: zoo
```

```
## Warning: package 'zoo' was built under R version 4.4.3
```

```
##
```

```
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      as.Date, as.Date.numeric
```

```
##
## ##### Warning from 'xts' package #####
## #
## # The dplyr lag() function breaks how base R's lag() function is supposed to #
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or #
## # source() into this session won't work correctly. #
## #
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop #
## # dplyr from breaking base R's lag() function. #
## #
## # Code in packages is not affected. It's protected by R's namespace mechanism #
## # Set 'options(xts.warn_dplyr_breaks_lag = FALSE)' to suppress this warning. #
## #
## #####
```

```
##
## Attaching package: 'xts'
```

```
## The following objects are masked from 'package:dplyr':
##
## first, last
```

```
## Loading required package: TTR
```

```
## Warning: package 'TTR' was built under R version 4.4.3
```

```
## Registered S3 method overwritten by 'quantmod':
## method from
## as.zoo.data.frame zoo
```

```
library(rugarch)
```

```
## Warning: package 'rugarch' was built under R version 4.4.3
```

```
## Loading required package: parallel
```

```
library(copula)
```

```
## Warning: package 'copula' was built under R version 4.4.3
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.2
```

```
library(reshape2)
```

```
## Warning: package 'reshape2' was built under R version 4.4.3
```

```
library(tidyr)
```

```
## Warning: package 'tidyr' was built under R version 4.4.2
```

```
##
```

```
## Attaching package: 'tidyr'
```

```
## The following object is masked from 'package:reshape2':
```

```
##
```

```
## smiths
```

```
# Step 2: Data Cleaning
```

```
options(repr.plot.width = 8, repr.plot.height = 5)
```

```
data <- read.csv('D:/statmethfindata.csv')
```

```
vars <- c("PlanName", "fy", "StateAbbrev",  
          "ActFundedRatio_GASB", "ActLiabilities_GASB", "ActAssets_GASB",  
          "EQTotal_Rtrn", "FITotal_Rtrn", "RETtotal_Rtrn", "PETTotal_Rtrn",  
          "EQTotal_Actl", "FITotal_Actl", "RETtotal_Actl", "PETTotal_Actl",  
          "contrib_tot", "expense_TotBenefits",  
          "InvestmentReturnAssumption_GASB",  
          "MktAssets_net")
```

```
df <- data[, vars]
```

```
df_clean <- df[!is.na(df$ActFundedRatio_GASB) &  
               !is.na(df$EQTotal_Rtrn) &  
               !is.na(df$FITotal_Rtrn), ]
```

```
# Step 3: Initial Asset Allocation
```

```
assets_annual <- df_clean %>%
```

```
  group_by(fy) %>%
```

```
  summarise(A_total = sum(ActAssets_GASB, na.rm = TRUE))
```

```
# Initial assets for each test year XX = total assets at end of XX-1
```

```
AO_list <- list()
```

```
for (XX in 2017:2024) {
```

```
  AO <- assets_annual %>% filter(fy == (XX - 1)) %>% pull(A_total)
```

```
  AO_list[[as.character(XX)]] <- AO
```

```
  cat("XX =", XX, " | Initial assets (XX-1):", format(AO, big.mark=","), "\n")
```

```
}
```

```
## XX = 2017 | Initial assets (XX-1): 3,466,769,903
```

```
## XX = 2018 | Initial assets (XX-1): 3,649,957,516
```

```
## XX = 2019 | Initial assets (XX-1): 3,855,423,375
```

```
## XX = 2020 | Initial assets (XX-1): 3,936,436,797
```

```
## XX = 2021 | Initial assets (XX-1): 4,112,889,948
```

```
## XX = 2022 | Initial assets (XX-1): 4,565,777,790
```

```
## XX = 2023 | Initial assets (XX-1): 4,805,627,648
```

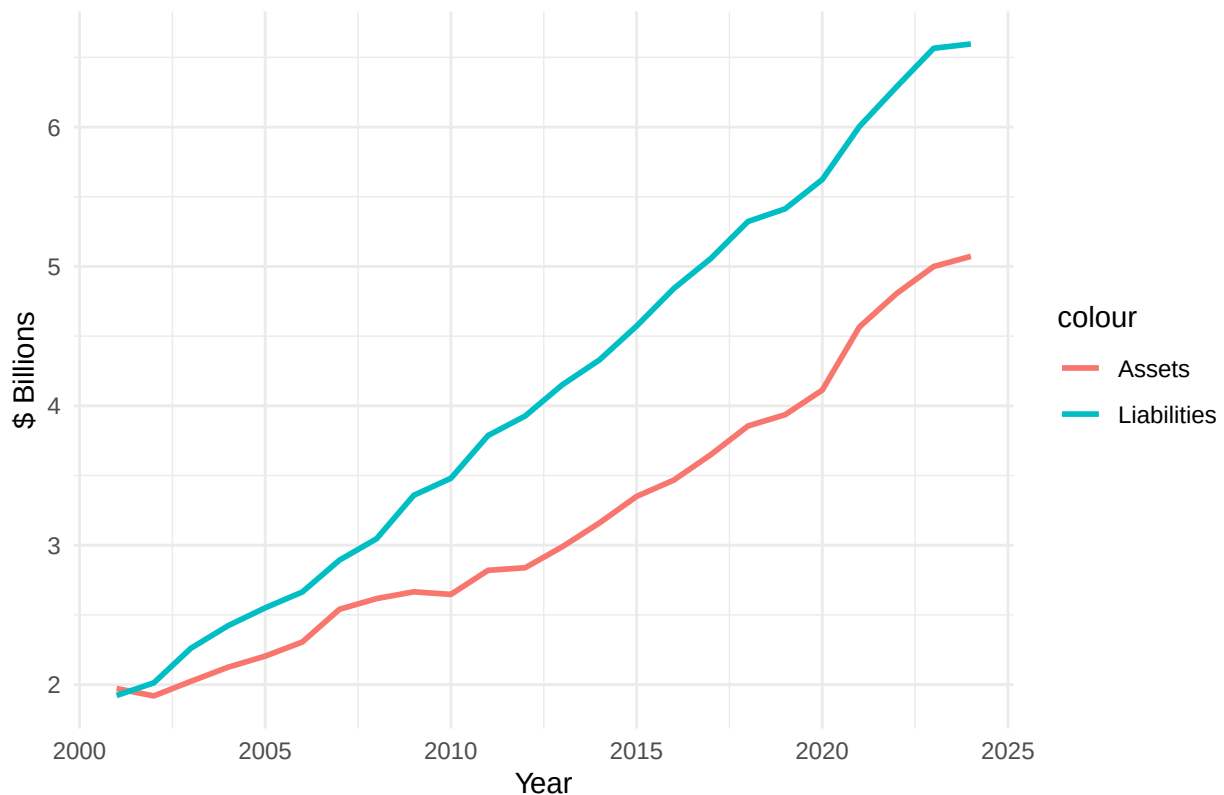
```
## XX = 2024 | Initial assets (XX-1): 4,999,830,852
```

```
agg_df <- df_clean %>%
  group_by(fy) %>%
  summarise(A = sum(ActAssets_GASB, na.rm=TRUE),
            L = sum(ActLiabilities_GASB, na.rm=TRUE),
            FR = A/L)

ggplot(agg_df, aes(x=fy)) +
  geom_line(aes(y=A/1e9, color="Assets"), size=1) +
  geom_line(aes(y=L/1e9, color="Liabilities"), size=1) +
  labs(title="Aggregate Sponsor Assets vs Liabilities",
       y="$ Billions", x="Year") +
  theme_minimal()
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

Aggregate Sponsor Assets vs Liabilities



```
# Step 4: Asset Returns
```

```
# Get monthly returns
```

```
tickers <- c("SPY", "IEF", "VNQ", "PSP")
```

```
getSymbols(tickers, src="yahoo", from="2006-01-01", to="2024-12-31", periodicity="monthly")
```

```
## [1] "SPY" "IEF" "VNQ" "PSP"
```

```
spy_ret <- diff(log(Ad(SPY)))
ief_ret <- diff(log(Ad(IEF)))
vnq_ret <- diff(log(Ad(VNQ)))
psp_ret <- diff(log(Ad(PSP)))

returns <- na.omit(merge(spy_ret, ief_ret, vnq_ret, psp_ret))
colnames(returns) <- c("EQ", "FI", "RE", "PE")

cat("Total return observations:", nrow(returns), "\n")
```

```
## Total return observations: 218
```

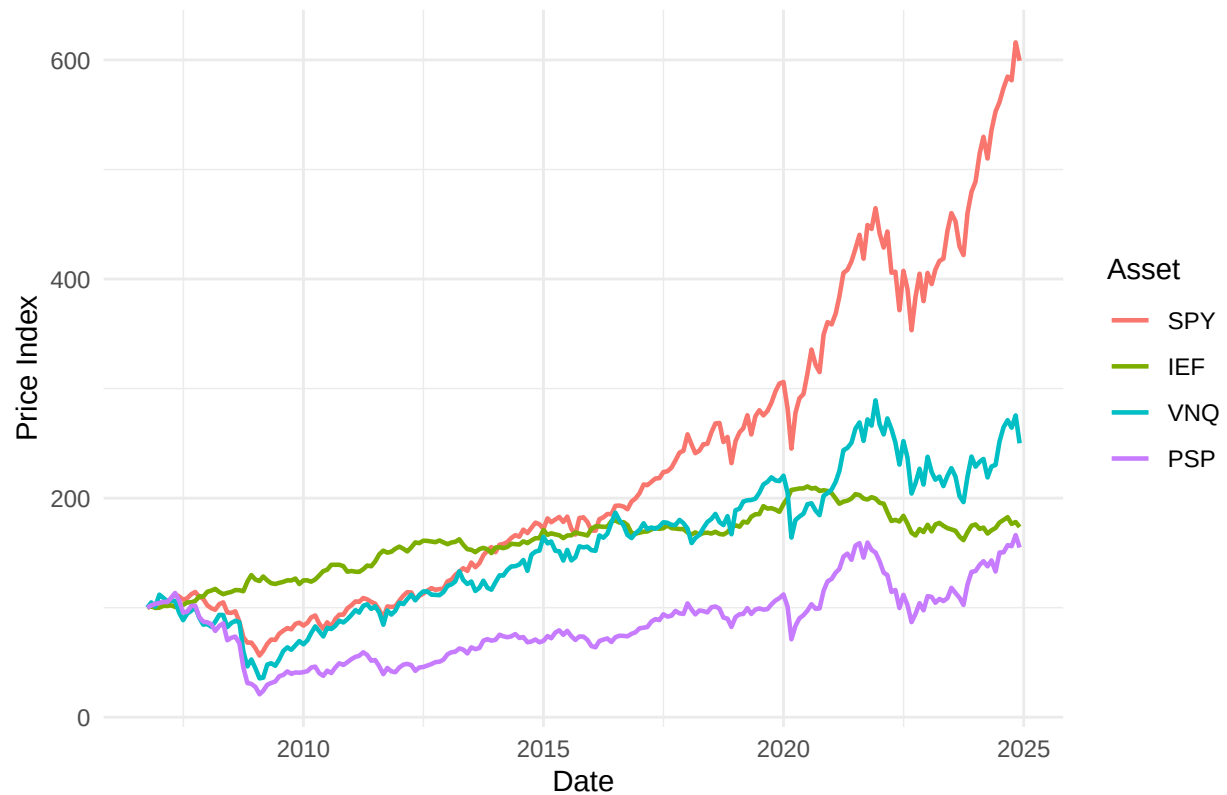
```
# Price growth
prices <- na.omit(merge(Ad(SPY), Ad(IEF), Ad(VNQ), Ad(PSP)))
colnames(prices) <- c("SPY", "IEF", "VNQ", "PSP")

prices_df <- data.frame(
  date = index(prices),
  SPY = as.numeric(prices$SPY) / as.numeric(prices$SPY)[1] * 100,
  IEF = as.numeric(prices$IEF) / as.numeric(prices$IEF)[1] * 100,
  VNQ = as.numeric(prices$VNQ) / as.numeric(prices$VNQ)[1] * 100,
  PSP = as.numeric(prices$PSP) / as.numeric(prices$PSP)[1] * 100
)

prices_long <- melt(prices_df, id.vars="date", variable.name="Asset", value.name="Price")

ggplot(prices_long, aes(x=date, y=Price, color=Asset)) +
  geom_line(size=0.8) +
  labs(title="Cumulative Price Growth (Normalized to 100)",
       x="Date", y="Price Index") +
  theme_minimal()
```

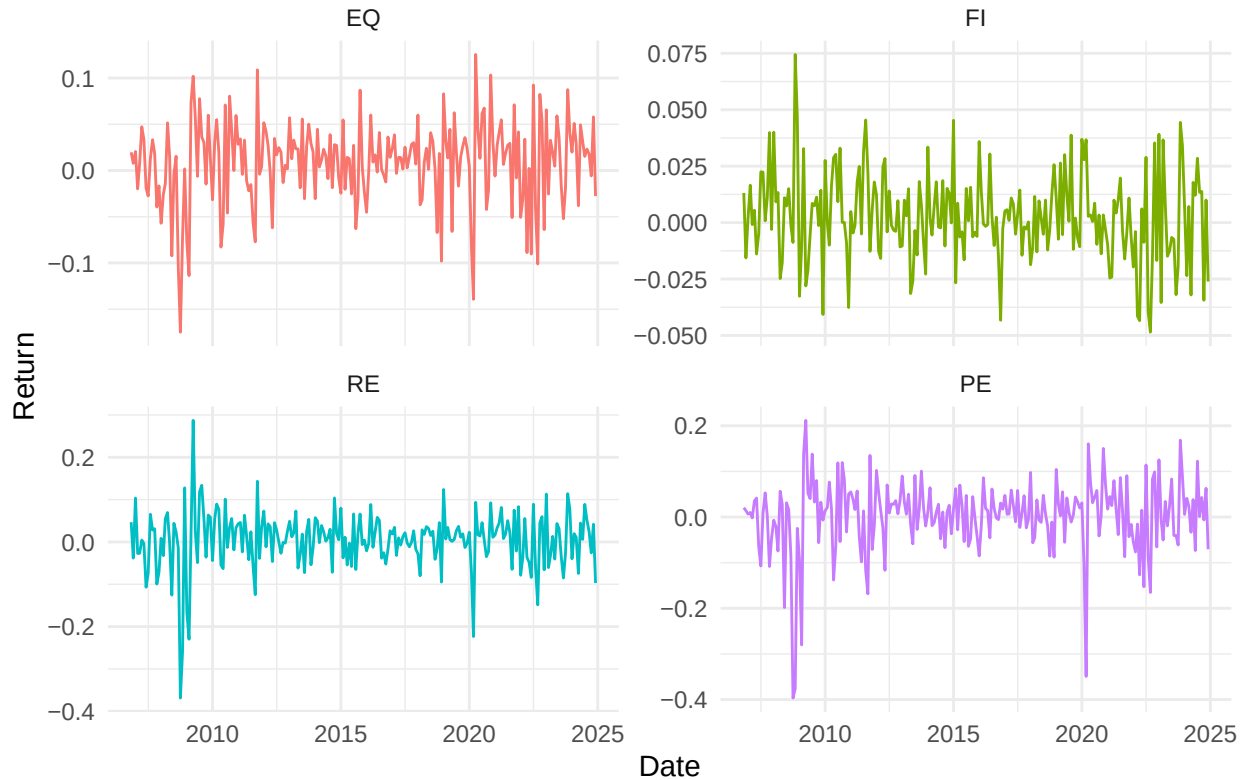
Cumulative Price Growth (Normalized to 100)



```
# Monthly returns
returns_df <- data.frame(date=index(returns), coredata(returns))
returns_long <- melt(returns_df, id.vars="date", variable.name="Asset", value.name="Return")

ggplot(returns_long, aes(x=date, y=Return, color=Asset)) +
  geom_line(size=0.5) +
  facet_wrap(~Asset, scales="free_y") +
  labs(title="Monthly Returns by Asset Class (2006-2024)",
       x="Date", y="Return") +
  theme_minimal() +
  theme(legend.position="none")
```

Monthly Returns by Asset Class (2006–2024)



```
# Step 5: GARCH

# GARCH spec
spec <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1,1)),
  mean.model = list(armaOrder = c(0,0), include.mean = TRUE),
  distribution.model = "sstd" # skewed student-t
)

# Fit GARCH per test year using expanding window
garch_fits_list <- list()

for (XX in 2017:2024) {
  end_date <- as.Date(paste0(XX-1, "-12-31"))
  ret_train <- returns[index(returns) <= end_date]

  fit_eq <- ugarchfit(spec, ret_train$EQ)
  fit_fi <- ugarchfit(spec, ret_train$FI)
  fit_re <- ugarchfit(spec, ret_train$RE)
  fit_pe <- ugarchfit(spec, ret_train$PE)

  garch_fits_list[[as.character(XX)]] <- list(
    EQ = fit_eq, FI = fit_fi, RE = fit_re, PE = fit_pe
  )

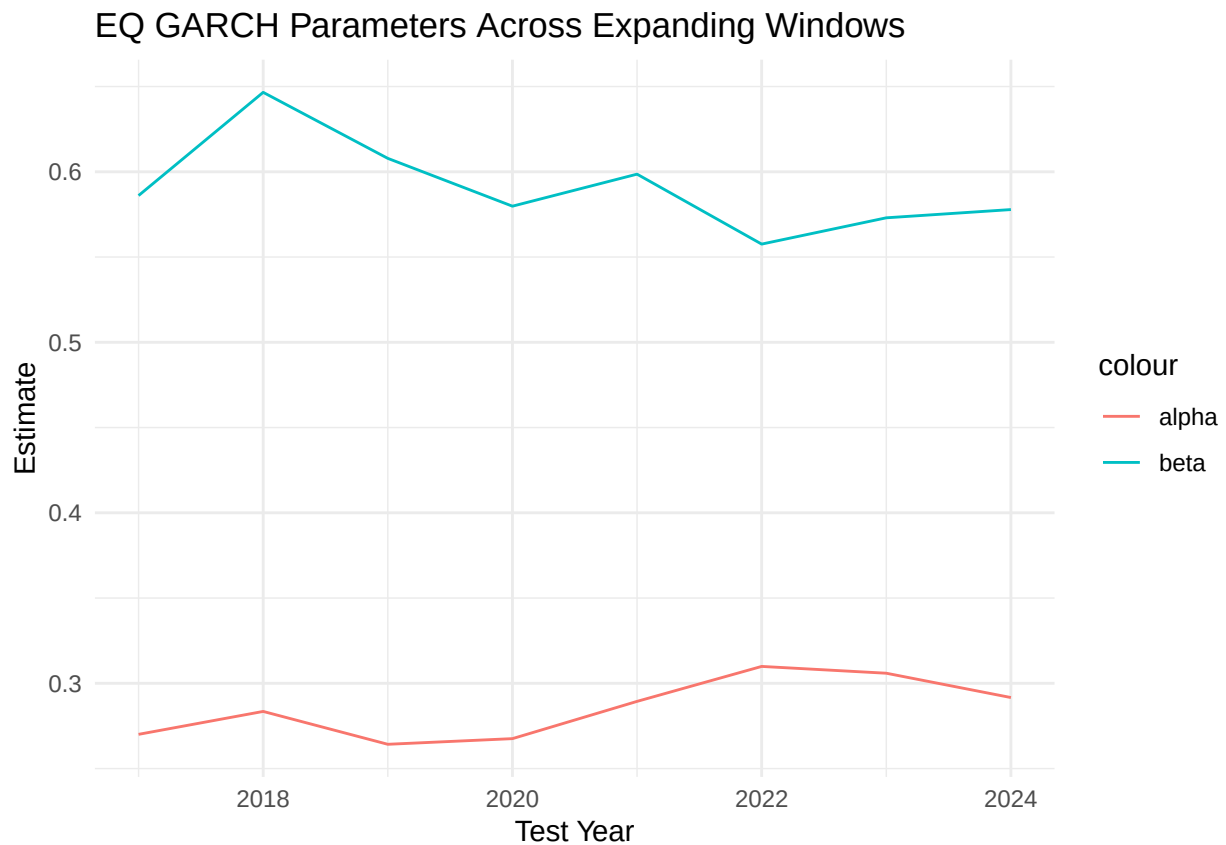
  cat("XX =", XX, " | GARCH fitted on", nrow(ret_train), "obs\n")
}
```

```
}
```

```
## XX = 2017 | GARCH fitted on 122 obs  
## XX = 2018 | GARCH fitted on 134 obs  
## XX = 2019 | GARCH fitted on 146 obs  
## XX = 2020 | GARCH fitted on 158 obs  
## XX = 2021 | GARCH fitted on 170 obs  
## XX = 2022 | GARCH fitted on 182 obs  
## XX = 2023 | GARCH fitted on 194 obs  
## XX = 2024 | GARCH fitted on 206 obs
```

```
# Step 6: GARCH Diagnostics
```

```
param_df <- data.frame(  
  year = 2017:2024,  
  alpha_EQ = sapply(garch_fits_list, function(f) coef(f$EQ)["alpha1"]),  
  beta_EQ = sapply(garch_fits_list, function(f) coef(f$EQ)["beta1"])  
)  
  
ggplot(param_df, aes(x=year)) +  
  geom_line(aes(y=alpha_EQ, color="alpha")) +  
  geom_line(aes(y=beta_EQ, color="beta")) +  
  labs(title="EQ GARCH Parameters Across Expanding Windows",  
        y="Estimate", x="Test Year") +  
  theme_minimal()
```



```

# For latest test year fits
fits_2024 <- garch_fits_list[["2024"]]

# Ljung-Box on standardized residuals - should be insignificant
for (asset in c("EQ", "FI", "RE", "PE")) {
  res <- residuals(fits_2024[[asset]], standardize=TRUE)
  lb <- Box.test(res, lag=10, type="Ljung-Box")
  lb_sq <- Box.test(res^2, lag=10, type="Ljung-Box")
  cat(asset, "| LB residuals p:", round(lb$p.value, 3),
      "| LB squared p:", round(lb_sq$p.value, 3), "\n")
}

```

```

## EQ | LB residuals p: 0.486 | LB squared p: 0.861
## FI | LB residuals p: 0.976 | LB squared p: 0.529
## RE | LB residuals p: 0.436 | LB squared p: 0.958
## PE | LB residuals p: 0.22 | LB squared p: 0.307

```

```

persistence_df <- data.frame(
  year = 2017:2024,
  EQ = apply(garch_fits_list, function(f) sum(coef(f$EQ)[c("alpha1", "beta1")])),
  FI = apply(garch_fits_list, function(f) sum(coef(f$FI)[c("alpha1", "beta1")])),
  RE = apply(garch_fits_list, function(f) sum(coef(f$RE)[c("alpha1", "beta1")])),
  PE = apply(garch_fits_list, function(f) sum(coef(f$PE)[c("alpha1", "beta1")])),
)
print(persistence_df)

```

```

##      year      EQ      FI      RE      PE
## 2017 2017 0.8561213 0.9988233 0.8922744 0.8774763
## 2018 2018 0.9301041 0.9962821 0.9602708 0.8698068
## 2019 2019 0.8720986 0.9959563 0.9492341 0.8462925
## 2020 2020 0.8473870 0.9751631 0.9504623 0.8455730
## 2021 2021 0.8880074 0.9791104 0.9445254 0.8764865
## 2022 2022 0.8674946 0.9722307 0.9138726 0.8405336
## 2023 2023 0.8789143 0.3298786 0.9147669 0.8565605
## 2024 2024 0.8694949 0.9253923 0.9101650 0.8487620

```

```

fits_2024 <- garch_fits_list[["2024"]]

end_date <- as.Date("2023-12-31")
ret_2024 <- returns[index(returns) <= end_date]

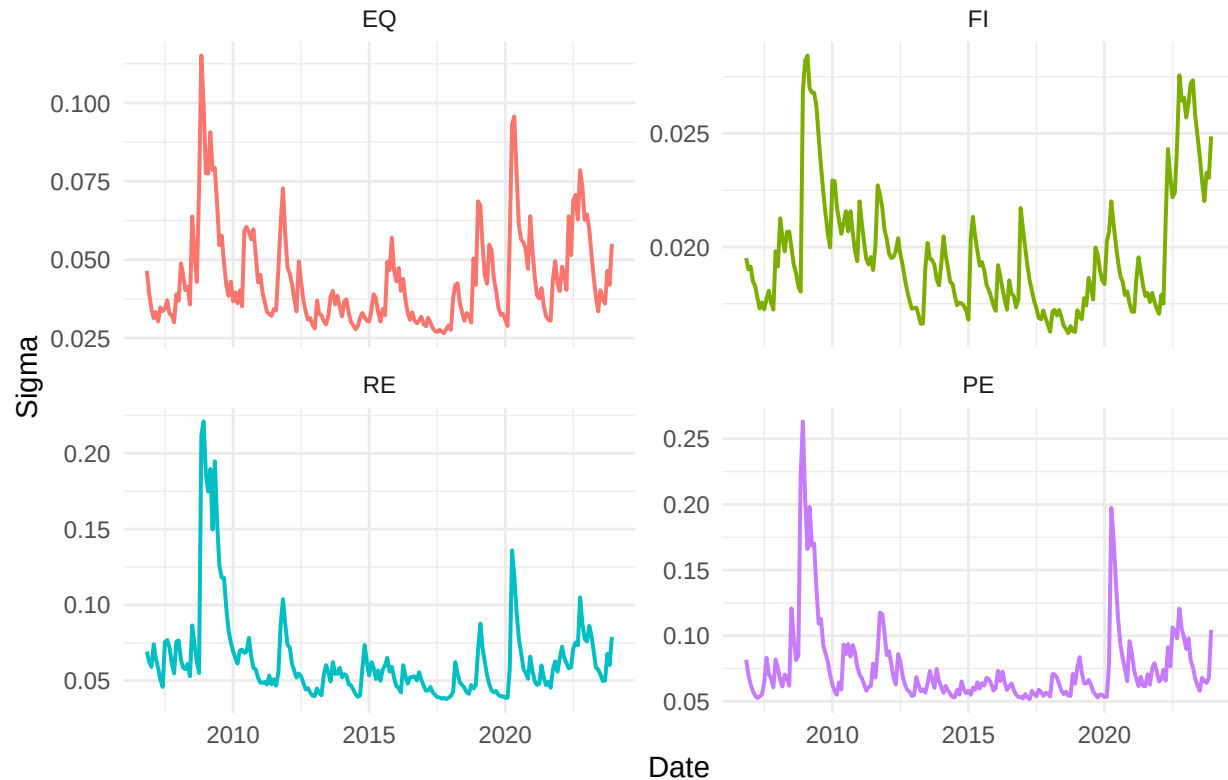
vol_df <- data.frame(
  date = index(ret_2024),
  EQ = as.numeric(sigma(fits_2024$EQ)),
  FI = as.numeric(sigma(fits_2024$FI)),
  RE = as.numeric(sigma(fits_2024$RE)),
  PE = as.numeric(sigma(fits_2024$PE))
)

vol_long <- melt(vol_df, id.vars="date", variable.name="Asset", value.name="Volatility")

```

```
ggplot(vol_long, aes(x=date, y=Volatility, color=Asset)) +
  geom_line(size=0.7) +
  facet_wrap(~Asset, scales="free_y") +
  labs(title="GARCH(1,1) Conditional Volatility (2006-2023)",
       x="Date", y="Sigma") +
  theme_minimal() +
  theme(legend.position="none")
```

GARCH(1,1) Conditional Volatility (2006–2023)

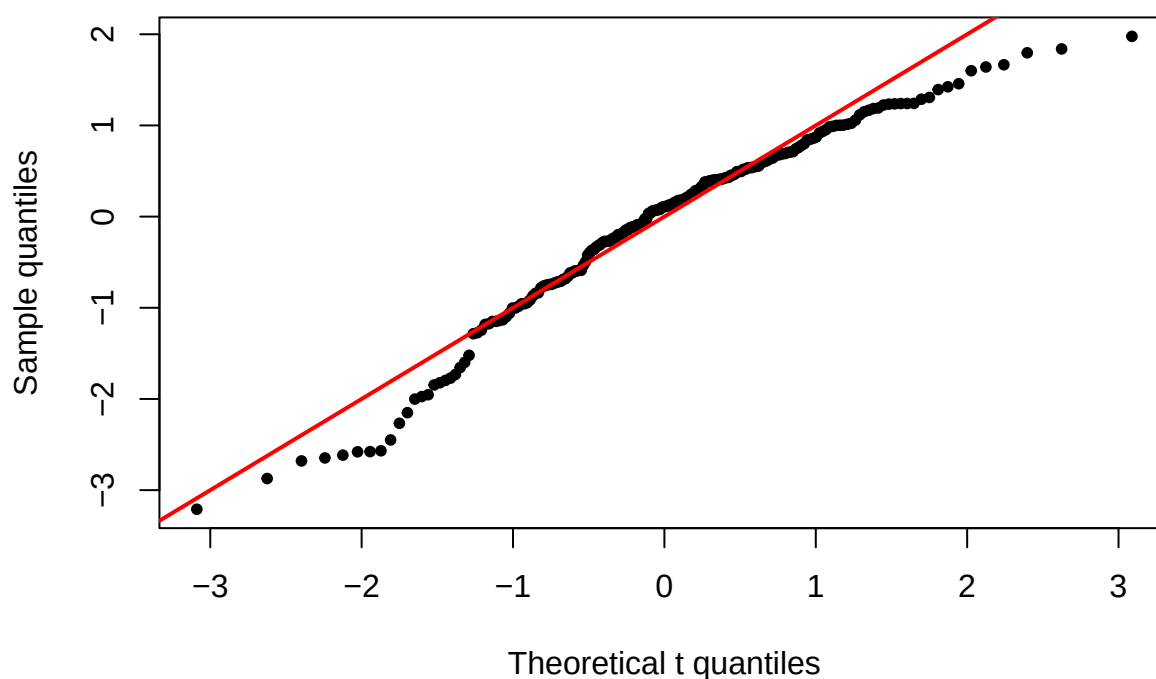


```
res <- as.numeric(residuals(fits_2024$EQ, standardize=TRUE))
shape <- coef(fits_2024$EQ)["shape"]

# Sort residuals and compute theoretical quantiles
n <- length(res)
theoretical <- qt(ppoints(n), df=shape)
sample_sorted <- sort(res)

plot(theoretical, sample_sorted,
     main="QQ Plot: EQ standardized residuals vs t-distribution",
     xlab="Theoretical t quantiles", ylab="Sample quantiles",
     pch=20)
abline(0, 1, col="red", lwd=2)
```

QQ Plot: EQ standardized residuals vs t-distribution



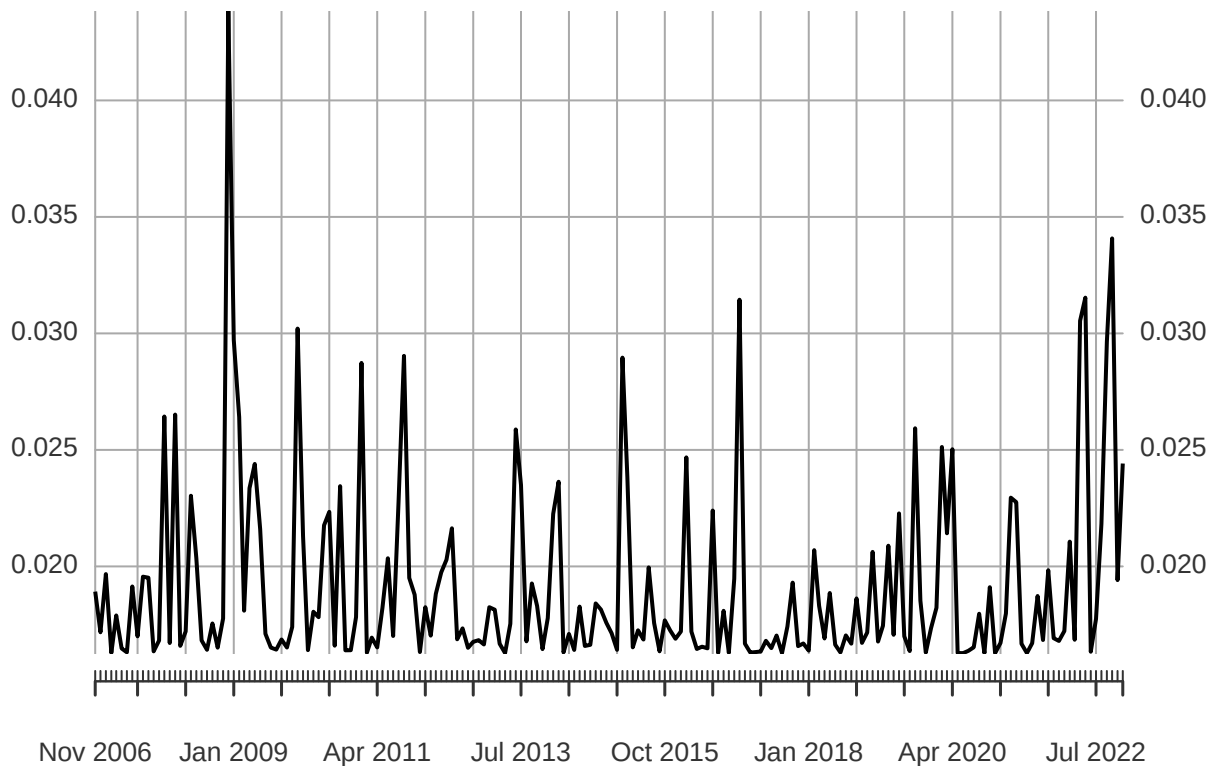
```
# Step 6b: Fixed Income 2023 Anomaly
fits_2023 <- garch_fits_list[["2023"]]
res <- residuals(fits_2023$FI, standardize=TRUE)
Box.test(res^2, lag=10, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: res^2
## X-squared = 5.1923, df = 10, p-value = 0.878
```

```
plot(sigma(fits_2023$FI), main="FI Conditional Vol - 2023 fit")
```

FI Conditional Vol – 2023 fit

2006-11-01 / 2022-12-01



```
# Step 7: Copula

copula_fits_list <- list()

for (XX in 2017:2024) {
  fits <- garch_fits_list[[as.character(XX)]]

  resid_eq <- as.numeric(residuals(fits$EQ, standardize=TRUE))
  resid_fi <- as.numeric(residuals(fits$FI, standardize=TRUE))
  resid_re <- as.numeric(residuals(fits$RE, standardize=TRUE))
  resid_pe <- as.numeric(residuals(fits$PE, standardize=TRUE))

  resid_matrix <- cbind(resid_eq, resid_fi, resid_re, resid_pe)
  u_matrix <- pobs(as.matrix(resid_matrix))

  t_cop <- tCopula(dim=4, dispstr="un")
  fit_cop <- fitCopula(t_cop, u_matrix, method="ml")

  copula_fits_list[[as.character(XX)]] <- fit_cop
}

cat("XX =", XX,
    "| EQ-FI:", round(fit_cop@estimate[1], 3),
    "| EQ-RE:", round(fit_cop@estimate[2], 3),
    "| EQ-PE:", round(fit_cop@estimate[3], 3),
    "| FI-RE:", round(fit_cop@estimate[4], 3),
    "| FI-PE:", round(fit_cop@estimate[5], 3),
```

```

    "| RE-PE:", round(fit_cop@estimate[6], 3),
    "| df:", round(fit_cop@estimate[7], 2), "\n")
}

```

```

## XX = 2017 | EQ-FI: -0.28 | EQ-RE: 0.662 | EQ-PE: 0.854 | FI-RE: 0.124 | FI-PE: -0.217 | RE-PE: 0.657
## XX = 2018 | EQ-FI: -0.266 | EQ-RE: 0.645 | EQ-PE: 0.832 | FI-RE: 0.14 | FI-PE: -0.199 | RE-PE: 0.631
## XX = 2019 | EQ-FI: -0.263 | EQ-RE: 0.628 | EQ-PE: 0.844 | FI-RE: 0.155 | FI-PE: -0.214 | RE-PE: 0.599
## XX = 2020 | EQ-FI: -0.282 | EQ-RE: 0.604 | EQ-PE: 0.85 | FI-RE: 0.175 | FI-PE: -0.219 | RE-PE: 0.586
## XX = 2021 | EQ-FI: -0.279 | EQ-RE: 0.63 | EQ-PE: 0.858 | FI-RE: 0.156 | FI-PE: -0.218 | RE-PE: 0.616
## XX = 2022 | EQ-FI: -0.255 | EQ-RE: 0.653 | EQ-PE: 0.848 | FI-RE: 0.137 | FI-PE: -0.196 | RE-PE: 0.63
## XX = 2023 | EQ-FI: -0.152 | EQ-RE: 0.678 | EQ-PE: 0.859 | FI-RE: 0.188 | FI-PE: -0.084 | RE-PE: 0.654
## XX = 2024 | EQ-FI: -0.096 | EQ-RE: 0.695 | EQ-PE: 0.853 | FI-RE: 0.237 | FI-PE: -0.021 | RE-PE: 0.673

```

```

plot_copula_evolution_gg <- function(pair_name, idx1, idx2) {
  all_data <- data.frame()

  for (XX in 2017:2024) {
    fits <- garch_fits_list[[as.character(XX)]]

    r1 <- as.numeric(residuals(fits[[idx1]], standardize=TRUE))
    r2 <- as.numeric(residuals(fits[[idx2]], standardize=TRUE))

    u1 <- rank(r1) / (length(r1) + 1)
    u2 <- rank(r2) / (length(r2) + 1)

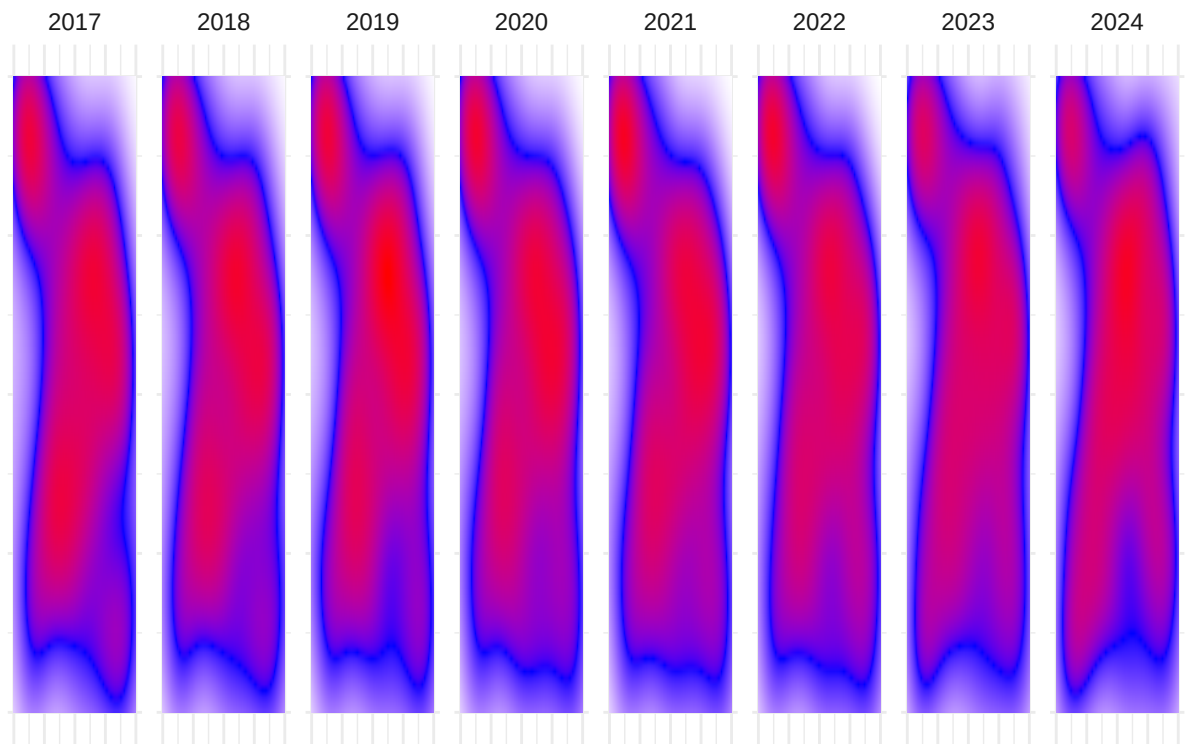
    all_data <- rbind(all_data, data.frame(u1=u1, u2=u2, year=XX))
  }

  ggplot(all_data, aes(x=u1, y=u2)) +
    stat_density_2d(aes(fill=after_stat(density)),
                    geom="raster", contour=FALSE) +
    scale_fill_gradientn(colors=c("white", "blue", "red")) +
    facet_wrap(~year, nrow=1) +
    labs(title=paste(pair_name, "Copula Density 2017-2024"),
         x="", y="") +
    theme_minimal() +
    theme(legend.position="none",
          axis.text=element_blank(),
          axis.ticks=element_blank())
}

plot_copula_evolution_gg("EQ-FI", "EQ", "FI")

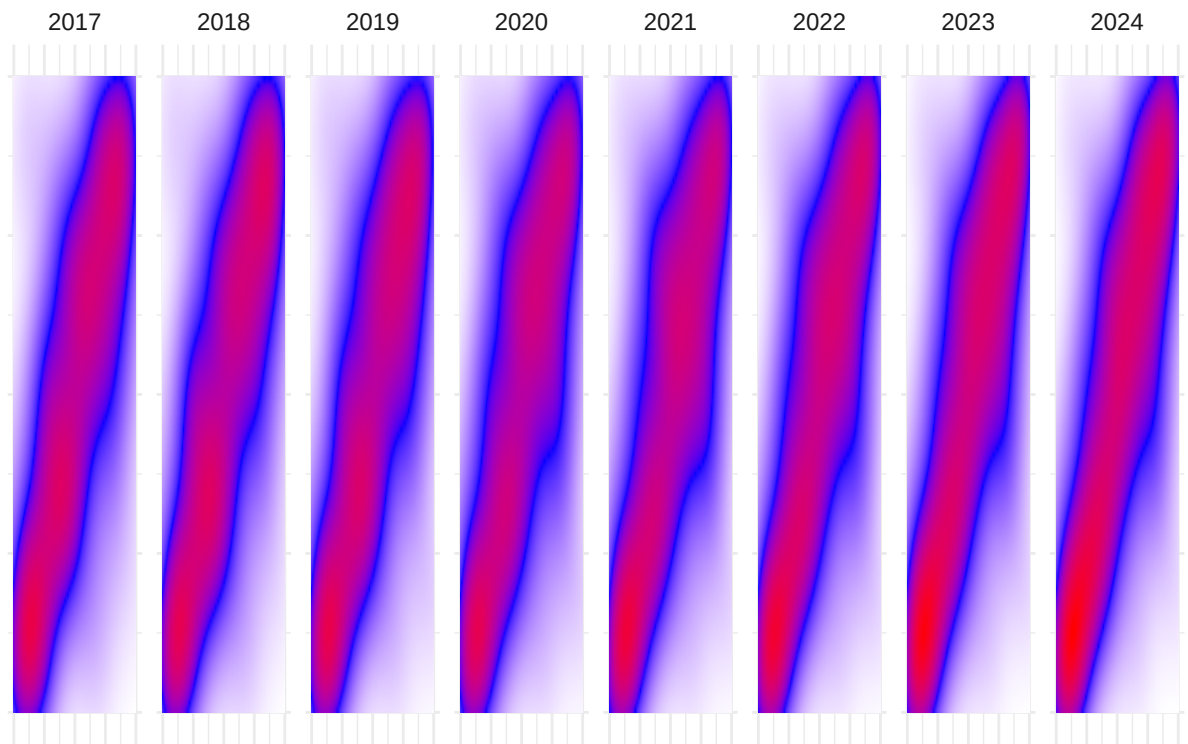
```

EQ-FI Copula Density 2017–2024



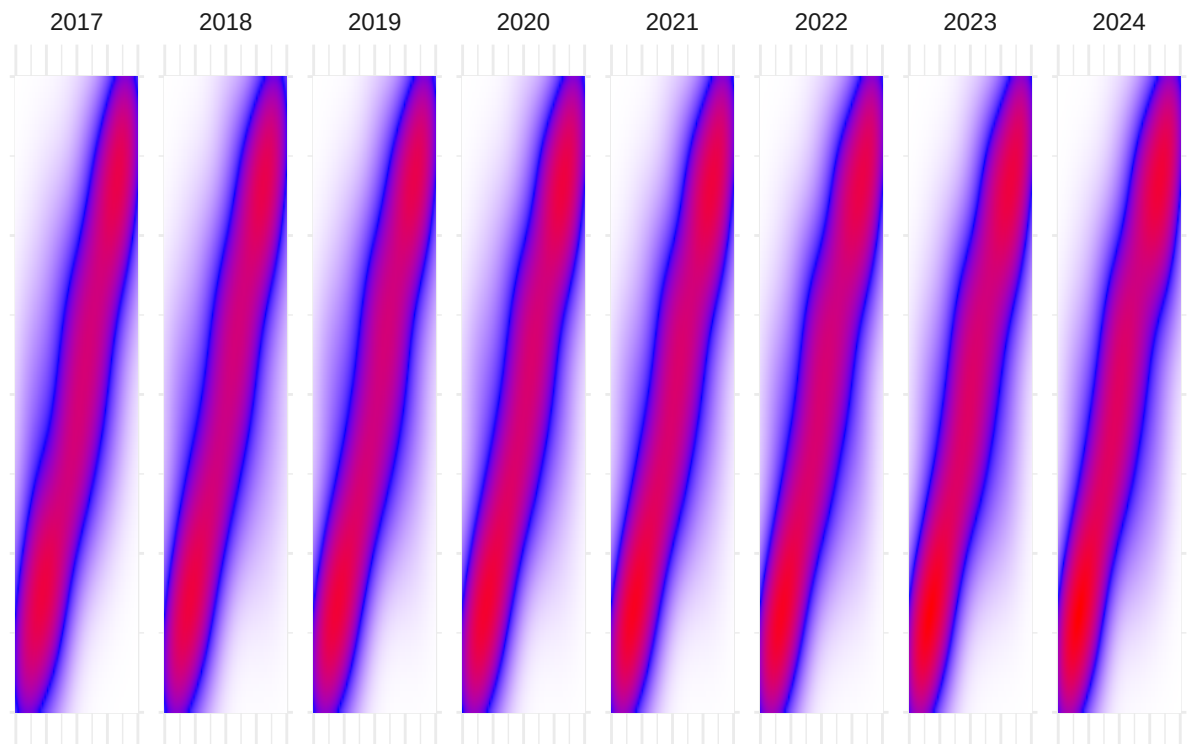
```
plot_copula_evolution_gg("EQ-RE", "EQ", "RE")
```

EQ-RE Copula Density 2017-2024



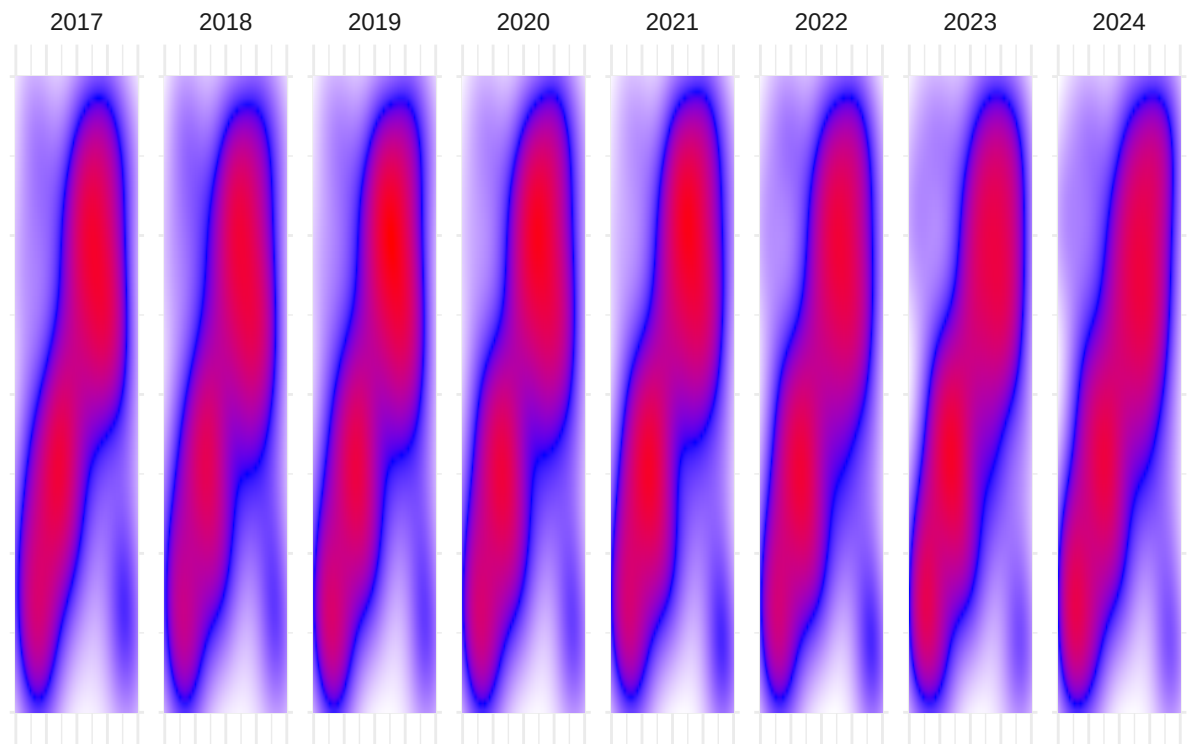
```
plot_copula_evolution_gg("EQ-PE", "EQ", "PE")
```

EQ-PE Copula Density 2017-2024



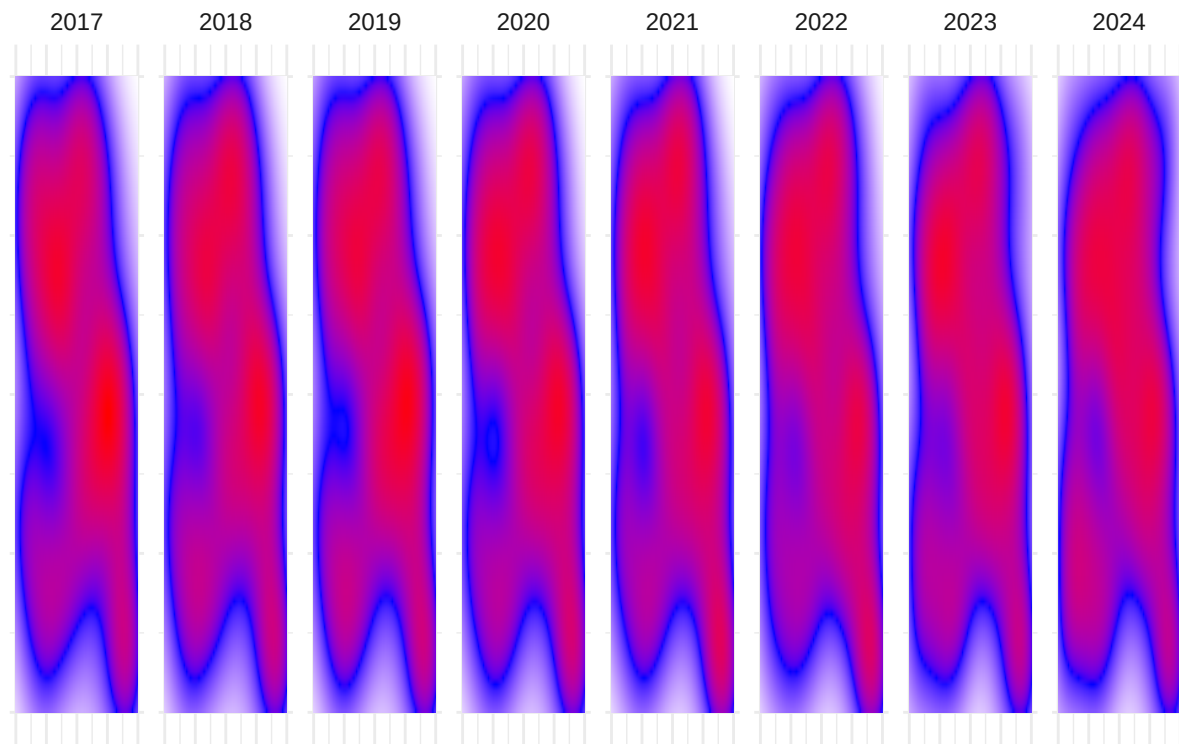
```
plot_copula_evolution_gg("FI-RE", "FI", "RE")
```

FI-RE Copula Density 2017–2024



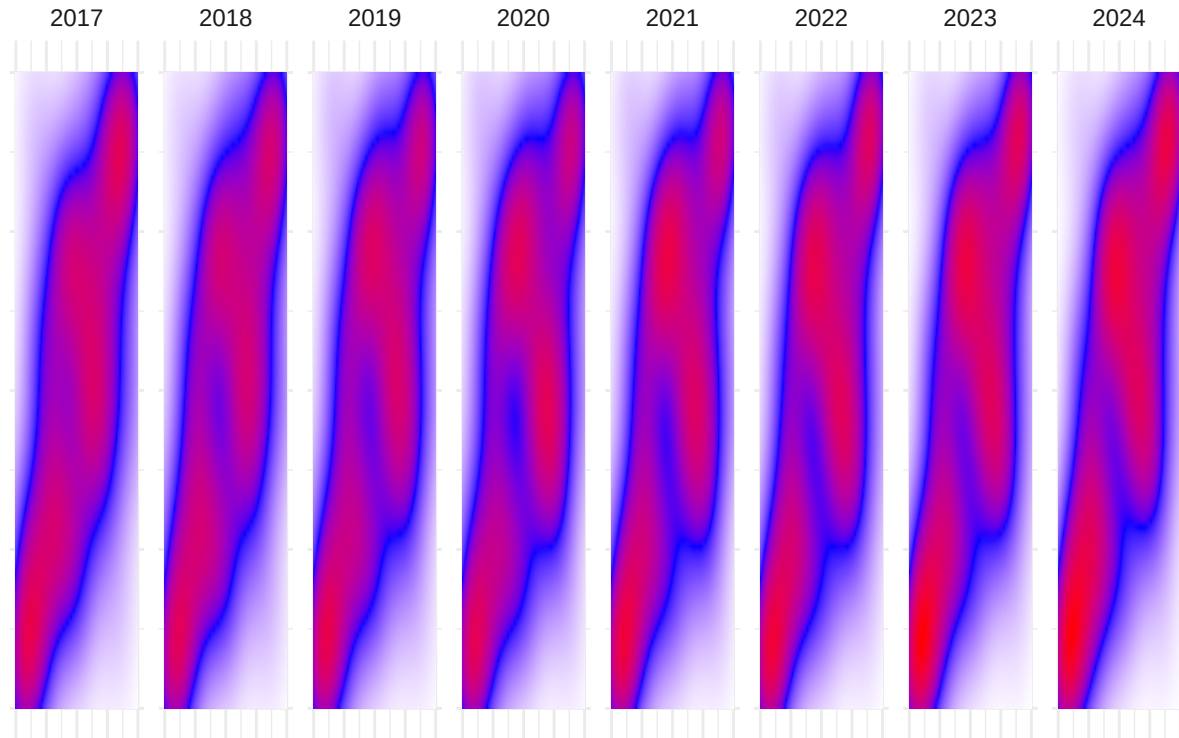
```
plot_copula_evolution_gg("FI-PE", "FI", "PE")
```

FI-PE Copula Density 2017–2024



```
plot_copula_evolution_gg("RE-PE", "RE", "PE")
```

RE-PE Copula Density 2017–2024



```
# Step 8: Weight Allocation
netflow_annual <- df_clean %>%
  group_by(fy) %>%
  summarise(
    net_flow = sum(contrib_tot + expense_TotBenefits, na.rm=TRUE)
  )

weights_annual <- df_clean %>%
  group_by(fy) %>%
  summarise(
    w_eq = mean(EQTotal_Act1, na.rm=TRUE),
    w_fi = mean(FITotal_Act1, na.rm=TRUE),
    w_re = mean(RETotal_Act1, na.rm=TRUE),
    w_pe = mean(PETotal_Act1, na.rm=TRUE)
  )

liab_annual <- df_clean %>%
  group_by(fy) %>%
  summarise(
    L_total = sum(ActLiabilities_GASB, na.rm=TRUE)
  )

print(netflow_annual)
```

```
## # A tibble: 24 x 2
##   fy    net_flow
```

```
##      <int>      <dbl>
## 1  2001 -30973222.
## 2  2002 -38628894.
## 3  2003 -39492056.
## 4  2004 -34685743.
## 5  2005 -43119873.
## 6  2006 -46833728.
## 7  2007 -52056226.
## 8  2008 -52081853.
## 9  2009 -60690173.
## 10 2010 -67890943.
## # i 14 more rows
```

```
print(weights_annual)
```

```
## # A tibble: 24 x 5
##       fy w_eq w_fi w_re w_pe
##   <int> <dbl> <dbl> <dbl> <dbl>
## 1  2001 0.565 0.329 0.0407 0.0331
## 2  2002 0.554 0.343 0.0418 0.0324
## 3  2003 0.579 0.314 0.0408 0.0316
## 4  2004 0.609 0.289 0.0393 0.0297
## 5  2005 0.599 0.295 0.0431 0.0312
## 6  2006 0.602 0.280 0.0498 0.0340
## 7  2007 0.593 0.273 0.0538 0.0382
## 8  2008 0.535 0.288 0.0670 0.0559
## 9  2009 0.516 0.293 0.0567 0.0586
## 10 2010 0.517 0.282 0.0526 0.0660
## # i 14 more rows
```

```
# Step 8: Volatility Clustering Simulation
set.seed(3)
N <- 10000
alpha_VaR <- 0.05
D_IEF <- 7.5
D_liab <- 13

# Get year-specific discount rate assumption from PPD
rate_annual <- df_clean %>%
  group_by(fy) %>%
  summarise(y0 = mean(InvestmentReturnAssumption_GASB, na.rm=TRUE))
end_date_full <- as.Date("2016-12-31")
ret_full <- returns[index(returns) <= end_date_full]
ief_annual <- apply.yearly(ret_full$FI, sum)
fi_bounds <- range(as.numeric(ief_annual))
cat("FI bounds:", fi_bounds, "\n")
```

```
## FI bounds: -0.06525999 0.1648171
```

```
results_list <- list()

for (XX in 2017:2024) {
```

```

# Get inputs for this year
# Net flow as % of starting assets
FR_prev <- agg_df %>% filter(fy == (XX-1)) %>% pull(FR)
L_prev <- liab_annual %>% filter(fy == (XX-1)) %>% pull(L_total)
AO <- FR_prev * L_prev
LO <- liab_annual %>% filter(fy == XX) %>% pull(L_total)
nf <- netflow_annual %>% filter(fy == XX) %>% pull(net_flow)

w <- weights_annual %>% filter(fy == XX) %>%
  dplyr::select(w_eq, w_fi, w_re, w_pe) %>% as.numeric()
w <- w / sum(w) # normalize to sum to 1

# Get GARCH fits and copula for this year
fits <- garch_fits_list[[as.character(XX)]]
cop_fit <- copula_fits_list[[as.character(XX)]]

# Extract GARCH parameters and last conditional variance
get_garch_params <- function(fit) {
  list(
    mu = coef(fit)["mu"],
    omega = coef(fit)["omega"],
    alpha = coef(fit)["alpha1"],
    beta = coef(fit)["beta1"],
    shape = coef(fit)["shape"],
    last_sigma2 = tail(sigma(fit), 1)^2,
    last_resid = tail(residuals(fit, standardize=FALSE), 1) # add this
  )
}

p_eq <- get_garch_params(fits$EQ)
p_fi <- get_garch_params(fits$FI)
p_re <- get_garch_params(fits$RE)
p_pe <- get_garch_params(fits$PE)

# Extract t-copula parameters
cop_params <- cop_fit@estimate
rho_mat <- matrix(c(
  1, cop_params[1], cop_params[2], cop_params[3],
  cop_params[1], 1, cop_params[4], cop_params[5],
  cop_params[2], cop_params[4], 1, cop_params[6],
  cop_params[3], cop_params[5], cop_params[6], 1
), nrow=4)
df_cop <- cop_params[7]

# Simulate N paths of 12 monthly returns
portfolio_returns <- numeric(N)
fi_annual_returns <- numeric(N)

for (i in 1:N) {
  # Simulate 12 months of correlated uniform innovations from t-copula
  u_sim <- rCopula(12, tCopula(param=cop_params[1:6], dim=4,
    dispstr="un", df=df_cop))

```

```

# Convert to t-distributed innovations
z <- qt(u_sim, df=df_cop)
z <- pmax(pmin(z, 5), -5)

# Simulate GARCH returns for each asset
sim_garch <- function(p, z_vec) {
  r <- numeric(12)
  sigma2 <- p$last_sigma2
  last_r <- as.numeric(p$last_resid) # actual last in-sample return
  for (t in 1:12) {
    prev_r <- if (t == 1) last_r else r[t-1]
    sigma2 <- p$omega + p$alpha * (prev_r - p$mu)^2 + p$beta * sigma2
    sigma2 <- min(sigma2, 1)
    r[t] <- p$mu + sqrt(sigma2) * z_vec[t]
  }
  r
}

r_eq <- sim_garch(p_eq, z[,1])
r_fi <- sim_garch(p_fi, z[,2])
r_re <- sim_garch(p_re, z[,3])
r_pe <- sim_garch(p_pe, z[,4])

# Annual log returns - sum monthly
lr_eq <- sum(r_eq)
lr_fi <- sum(r_fi)
lr_re <- sum(r_re)
lr_pe <- sum(r_pe)

# Convert to simple returns
sr <- exp(c(lr_eq, lr_fi, lr_re, lr_pe)) - 1

# Portfolio return using normalized weights
portfolio_returns[i] <- sum(w * sr)
fi_annual_returns[i] <- sr[2] # FI return for rate adjustment
}

# Convert to asset values and add net flows
portfolio_returns <- pmax(pmin(portfolio_returns, 10), -1)
A_paths <- A0 * (1 + portfolio_returns) + nf

# Winsorize FI returns at historical bounds before rate adjustment
fi_bounds <- range(as.numeric(ief_annual))
fi_annual_returns_capped <- pmax(pmin(fi_annual_returns, fi_bounds[2]), fi_bounds[1])

# Revalue liabilities using implied rate change from FI return
y0 <- rate_annual %>% filter(fy == XX) %>% pull(y0) # base discount rate
delta_y <- -fi_annual_returns_capped / D_IEF
L_paths <- L0 * ((1 + y0) / (1 + y0 + delta_y))^D_liab

# Funded ratio per path
FR_paths <- A_paths / L_paths

```

```

# VaR and CVaR at 5%
VaR_95 <- quantile(FR_paths, alpha_VaR)
CVaR_95 <- mean(FR_paths[FR_paths <= VaR_95])

# Actual reported funded ratio
FR_actual <- agg_df %>% filter(fy == XX) %>% pull(FR)

results_list[[as.character(XX)]] <- data.frame(
  year = XX,
  VaR_95 = VaR_95,
  CVaR_95 = CVaR_95,
  FR_mean = mean(FR_paths),
  FR_actual = FR_actual
)
}

# Step 9: Constant Variance Simulation
set.seed(3)
results_const_list <- list()

for (XX in 2017:2024) {
  FR_prev <- agg_df %>% filter(fy == (XX-1)) %>% pull(FR)
  L_prev <- liab_annual %>% filter(fy == (XX-1)) %>% pull(L_total)
  A0 <- FR_prev * L_prev
  L0 <- liab_annual %>% filter(fy == XX) %>% pull(L_total)
  nf <- netflow_annual %>% filter(fy == XX) %>% pull(net_flow)

  w <- weights_annual %>% filter(fy == XX) %>%
    dplyr::select(w_eq, w_fi, w_re, w_pe) %>% as.numeric()
  w <- w / sum(w)

  end_date <- as.Date(paste0(XX-1, "-12-31"))
  ret_train <- returns[index(returns) <= end_date]
  mu_const <- colMeans(ret_train)
  sd_const <- apply(ret_train, 2, sd)

  portfolio_returns <- numeric(N)
  fi_annual_returns <- numeric(N)

  for (i in 1:N) {
    r_mat <- matrix(rnorm(12 * 4), nrow=12)
    r_mat <- sweep(r_mat, 2, sd_const, "*")
    r_mat <- sweep(r_mat, 2, mu_const, "+")
    sr <- exp(colSums(r_mat)) - 1
    portfolio_returns[i] <- sum(w * sr)
    fi_annual_returns[i] <- sr[2]
  }

  portfolio_returns <- pmax(pmin(portfolio_returns, 10), -1)
  fi_annual_returns_capped <- pmax(pmin(fi_annual_returns, fi_bounds[2]), fi_bounds[1])

  A_paths <- A0 * (1 + portfolio_returns) + nf

```

```

y0 <- rate_annual %>% filter(fy == XX) %>% pull(y0)
delta_y <- -fi_annual_returns_capped / D_IEF
L_paths <- L0 * ((1 + y0) / (1 + y0 + delta_y))^D_liab
FR_paths <- A_paths / L_paths

VaR_95 <- quantile(FR_paths, alpha_VaR)
CVaR_95 <- mean(FR_paths[FR_paths <= VaR_95])
FR_actual <- agg_df %>% filter(fy == XX) %>% pull(FR)

results_const_list[[as.character(XX)]] <- data.frame(
  year = XX,
  VaR_95_const = VaR_95,
  CVaR_95_const = CVaR_95,
  FR_mean_const = mean(FR_paths),
  FR_actual = FR_actual
)

cat("XX =", XX,
    "| VaR(const):", round(VaR_95, 4),
    "| CVaR(const):", round(CVaR_95, 4), "\n")
}

```

```

## XX = 2017 | VaR(const): 0.529 | CVaR(const): 0.5061
## XX = 2018 | VaR(const): 0.5386 | CVaR(const): 0.5149
## XX = 2019 | VaR(const): 0.559 | CVaR(const): 0.5353
## XX = 2020 | VaR(const): 0.558 | CVaR(const): 0.535
## XX = 2021 | VaR(const): 0.5389 | CVaR(const): 0.5149
## XX = 2022 | VaR(const): 0.5872 | CVaR(const): 0.5591
## XX = 2023 | VaR(const): 0.5849 | CVaR(const): 0.5551
## XX = 2024 | VaR(const): 0.6086 | CVaR(const): 0.5791

```

```

# Step 10: Var/cVaR Comparison
results_df <- do.call(rbind, results_list)
results_const_df <- do.call(rbind, results_const_list)

# Combine results
comparison_df <- merge(
  results_df[, c("year", "VaR_95", "CVaR_95")],
  results_const_df[, c("year", "VaR_95_const", "CVaR_95_const")],
  by = "year"
)

print(comparison_df)

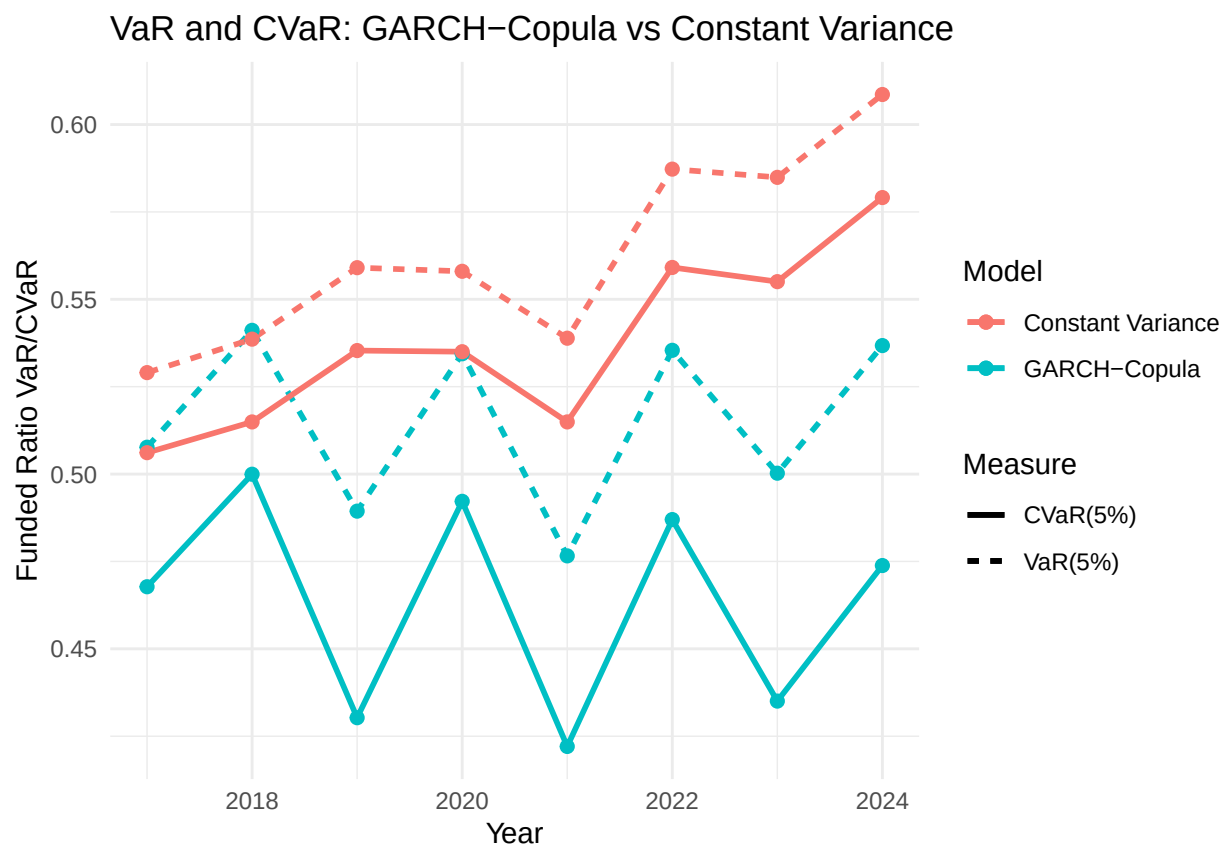
```

```

##   year   VaR_95  CVaR_95 VaR_95_const CVaR_95_const
## 1 2017 0.5076796 0.4677590   0.5290287   0.5061204
## 2 2018 0.5411207 0.4999531   0.5386022   0.5149162
## 3 2019 0.4893923 0.4302887   0.5590485   0.5353369
## 4 2020 0.5344179 0.4922033   0.5580151   0.5350109
## 5 2021 0.4765913 0.4220638   0.5388707   0.5149222
## 6 2022 0.5354033 0.4869831   0.5872436   0.5591154
## 7 2023 0.5002477 0.4350431   0.5848925   0.5550536
## 8 2024 0.5367631 0.4738311   0.6085983   0.5791103

```

```
# Plot
ggplot(comparison_df, aes(x=year)) +
  geom_line(aes(y=VaR_95, color="GARCH-Copula", linetype="VaR(5%)"), size=1) +
  geom_line(aes(y=CVaR_95, color="GARCH-Copula", linetype="CVaR(5%)"), size=1) +
  geom_line(aes(y=VaR_95_const, color="Constant Variance", linetype="VaR(5%)"), size=1) +
  geom_line(aes(y=CVaR_95_const, color="Constant Variance", linetype="CVaR(5%)"), size=1) +
  geom_point(aes(y=VaR_95, color="GARCH-Copula"), size=2) +
  geom_point(aes(y=CVaR_95, color="GARCH-Copula"), size=2) +
  geom_point(aes(y=VaR_95_const, color="Constant Variance"), size=2) +
  geom_point(aes(y=CVaR_95_const, color="Constant Variance"), size=2) +
  labs(title="VaR and CVaR: GARCH-Copula vs Constant Variance",
       x="Year", y="Funded Ratio VaR/CVaR",
       color="Model", linetype="Measure") +
  theme_minimal()
```



```
comparison_df$VaR_gap <- comparison_df$VaR_95_const - comparison_df$VaR_95
comparison_df$CVaR_gap <- comparison_df$CVaR_95_const - comparison_df$CVaR_95
cat("Mean VaR gap:", round(mean(comparison_df$VaR_gap), 4), "\n")
```

```
## Mean VaR gap: 0.0478
```

```
cat("Mean CVaR gap:", round(mean(comparison_df$CVaR_gap), 4), "\n")
```

```
## Mean CVaR gap: 0.0739
```

```
cat("Max VaR gap:", round(max(comparison_df$VaR_gap), 4), "\n")
```

```
## Max VaR gap: 0.0846
```

```
cat("Max CVaR gap:", round(max(comparison_df$CVaR_gap), 4), "\n")
```

```
## Max CVaR gap: 0.12
```

```
# Step 11: 2021 Market Crash Predictions
# FR distribution comparison for 2021
set.seed(3)
XX <- 2021

# GARCH-Copula paths
FR_prev <- agg_df %>% filter(fy == (XX-1)) %>% pull(FR)
L_prev <- liab_annual %>% filter(fy == (XX-1)) %>% pull(L_total)
A0 <- FR_prev * L_prev
L0 <- liab_annual %>% filter(fy == XX) %>% pull(L_total)
nf <- netflow_annual %>% filter(fy == XX) %>% pull(net_flow)

w <- weights_annual %>% filter(fy == XX) %>%
  dplyr::select(w_eq, w_fi, w_re, w_pe) %>% as.numeric()
w <- w / sum(w)

fits <- garch_fits_list[[as.character(XX)]]
cop_fit <- copula_fits_list[[as.character(XX)]]
cop_params <- cop_fit@estimate
df_cop <- cop_params[7]

p_eq <- get_garch_params(fits$EQ)
p_fi <- get_garch_params(fits$FI)
p_re <- get_garch_params(fits$RE)
p_pe <- get_garch_params(fits$PE)

portfolio_returns_garch <- numeric(N)
fi_returns_garch <- numeric(N)

for (i in 1:N) {
  u_sim <- rCopula(12, tCopula(param=cop_params[1:6], dim=4, dispstr="un", df=df_cop))
  z <- qt(u_sim, df=df_cop)
  z <- pmax(pmin(z, 5), -5)
  r_eq <- sim_garch(p_eq, z[,1])
  r_fi <- sim_garch(p_fi, z[,2])
  r_re <- sim_garch(p_re, z[,3])
  r_pe <- sim_garch(p_pe, z[,4])
  sr <- exp(c(sum(r_eq), sum(r_fi), sum(r_re), sum(r_pe))) - 1
  portfolio_returns_garch[i] <- sum(w * sr)
  fi_returns_garch[i] <- sr[2]
}

portfolio_returns_garch <- pmax(pmin(portfolio_returns_garch, 10), -1)
fi_returns_garch_capped <- pmax(pmin(fi_returns_garch, fi_bounds[2]), fi_bounds[1])
```

```

y0 <- rate_annual %>% filter(fy == XX) %>% pull(y0)
delta_y <- -fi_returns_garch_capped / D_IEF
L_paths_garch <- L0 * ((1 + y0) / (1 + y0 + delta_y))^D_liab
A_paths_garch <- A0 * (1 + portfolio_returns_garch) + nf
FR_garch <- A_paths_garch / L_paths_garch

# Constant variance paths
end_date <- as.Date(paste0(XX-1, "-12-31"))
ret_train <- returns[index(returns) <= end_date]
mu_const <- colMeans(ret_train)
sd_const <- apply(ret_train, 2, sd)

portfolio_returns_const <- numeric(N)
fi_returns_const <- numeric(N)

for (i in 1:N) {
  r_mat <- matrix(rnorm(12 * 4), nrow=12)
  r_mat <- sweep(r_mat, 2, sd_const, "*")
  r_mat <- sweep(r_mat, 2, mu_const, "+")
  sr <- exp(colSums(r_mat)) - 1
  portfolio_returns_const[i] <- sum(w * sr)
  fi_returns_const[i] <- sr[2]
}

portfolio_returns_const <- pmax(pmin(portfolio_returns_const, 10), -1)
fi_returns_const_capped <- pmax(pmin(fi_returns_const, fi_bounds[2]), fi_bounds[1])
delta_y_const <- -fi_returns_const_capped / D_IEF
L_paths_const <- L0 * ((1 + y0) / (1 + y0 + delta_y_const))^D_liab
A_paths_const <- A0 * (1 + portfolio_returns_const) + nf
FR_const <- A_paths_const / L_paths_const

# Combine and plot
dist_df <- data.frame(
  FR = c(FR_garch, FR_const),
  Model = rep(c("GARCH-Copula", "Constant Variance"), each=N)
)

VaR_g <- quantile(FR_garch, 0.05)
VaR_c <- quantile(FR_const, 0.05)
CVaR_g <- mean(FR_garch[FR_garch <= VaR_g])
CVaR_c <- mean(FR_const[FR_const <= VaR_c])

dist_df_trimmed <- dist_df %>% filter(FR >= 0.0 & FR <= quantile(dist_df$FR, 0.995))

ggplot(dist_df_trimmed, aes(x=FR, fill=Model, color=Model)) +
  geom_density(alpha=0.3, size=0.8) +
  geom_vline(xintercept=VaR_g, color="#00BFC4", linetype="dashed", size=1) +
  geom_vline(xintercept=VaR_c, color="#F8766D", linetype="dashed", size=1) +
  geom_vline(xintercept=CVaR_g, color="#00BFC4", linetype="solid", size=1) +
  geom_vline(xintercept=CVaR_c, color="#F8766D", linetype="solid", size=1) +
  labs(title="Funded Ratio Distribution: 2021",
       subtitle="Dashed = VaR(5%), Solid = CVaR(5%)",

```

```
x="Funded Ratio", y="Density") +  
theme_minimal()
```

Funded Ratio Distribution: 2021

Dashed = VaR(5%), Solid = CVaR(5%)

